

CNN Accuracy vs. Compute

The Input-Resolution Trade-off

Same CNN architecture. Same dataset (CIFAR-10). Only the input resolution changes: $32 \times 32 \rightarrow 64 \times 64 \rightarrow 128 \times 128$.

1 Motivation

Resolution is usually picked ad hoc. But it drives a direct accuracy $\leftrightarrow$ compute trade-off that matters for edge devices, latency budgets, and training cost.

2 Main Idea

Train one fixed CNN architecture on CIFAR-10 resized to 32, 64, and 128 px. GlobalAveragePooling keeps parameter count constant, so resolution is the only variable.

3 Result

Compute scaled roughly as expected ($\sim 4\times$ per doubling). But accuracy did NOT rise with resolution — it fell. Root cause: a fixed-depth network's receptive field covers a shrinking fraction of a larger image.

Related Work

How resolution, receptive field, and model capacity interact

1

EfficientNet: Rethinking Model Scaling for CNNs

Tan & Le · ICML 2019

Depth, width, and resolution must scale together (compound scaling). Raising resolution alone, without matching capacity, is a known sub-optimal regime.

arxiv.org/abs/1905.11946

2

Fixing the Train-Test Resolution Discrepancy

Touvron, Vedaldi, Douze & Jégou · NeurIPS 2019

A mismatch between train/test resolution changes the effective object scale the network sees, and can hurt accuracy if not corrected.

arxiv.org/abs/1906.06423

3

Understanding the Effective Receptive Field in Deep CNNs

Luo, Li, Urtasun & Zemel · NeurIPS 2016

A network's effective receptive field grows only with the square root of depth — a fixed-depth net does not “see” proportionally more of a larger image.

proceedings.neurips.cc/paper/2016 (Abstract page)

4

Enhancing Sensor Resolution Improves CNN Accuracy Given the Same Parameters/FLOPs

Borji · 2021 (incl. CIFAR-10)

Confirms directly on CIFAR-10: higher resolution only helps once the architecture's capacity is adjusted to match — naive upscaling can hurt accuracy.

arxiv.org/abs/2103.05251

Approach

Controlling everything except resolution

1

Fix dataset & architecture

CIFAR-10 (10 classes), identical 3-block CNN (Conv+MaxPool × 3) for every run.

2

Resize inputs

Bilinear resize (`tf.image.resize`) to 32×32, 64×64, and 128×128 — same images, different scale.

3

Neutralize the parameter-count trap

GlobalAveragePooling2D instead of Flatten, so dense-layer size (and total params) stays constant across resolutions.

4

Match every hyperparameter

Same optimizer (Adam), loss, batch size, and epoch count for all three resolutions.

5

Measure three things

Test accuracy, wall-clock training time, and per-image inference time, for each resolution.

Links Used in This Approach

- **CIFAR-10 dataset (Keras)**

keras.io/api/datasets/cifar10

- **`tf.image.resize` docs**

tensorflow.org/api_docs/python/tf/image/resize

- **GlobalAveragePooling2D docs**

keras.io/api/layers/pooling_layers/global_average_pooling2d

- **EfficientNet (compound scaling)**

arxiv.org/abs/1905.11946

- **Effective Receptive Field**

proceedings.neurips.cc/paper/2016

- **Sensor-resolution CNN study**

arxiv.org/abs/2103.05251

Code

Everything above is reproducible end-to-end in one script

`cnn_resolution_experiment.py`

Google-Colab-ready TensorFlow/Keras script: loads CIFAR-10, resizes to 32/64/128 px, builds the fixed-capacity CNN (GlobalAveragePooling), trains and times all three resolutions, then plots accuracy, training time, and inference time.

Shared as a downloadable file earlier in this conversation — open it in Google Colab (Runtime → Change runtime type → GPU) and run top to bottom.

3

resolutions compared

~4x

compute per doubling

1

fixed architecture